Regression and Classification with MultiArchitecture (Deep) Neural Networks

Quarto

```r
set.seed(260408)
```

```r
reticulate::conda_list()
```

```
                    name
1                   base
2 earth-analytics-python
3               img_proj
4       python-analytics
5           r-tensorflow
                                                                  python
1                                C:\\Users\\stvya\\anaconda3/python.exe
2 C:\\Users\\stvya\\anaconda3\\envs\\earth-analytics-python/python.exe
3               C:\\Users\\stvya\\anaconda3\\envs\\img_proj/python.exe
4       C:\\Users\\stvya\\anaconda3\\envs\\python-analytics/python.exe
5           C:\\Users\\stvya\\anaconda3\\envs\\r-tensorflow/python.exe
```

```r
library(reticulate)
use_condaenv("r-tensorflow", required = TRUE)
```

```r
py_config()
```

```
python:         C:/Users/stvya/anaconda3/envs/r-tensorflow/python.exe
libpython:      C:/Users/stvya/anaconda3/envs/r-tensorflow/python310.dll
pythonhome:     C:/Users/stvya/anaconda3/envs/r-tensorflow
```

```
version:        3.10.19 | packaged by conda-forge | (main, Jan 26 2026, 23:39:36) [MSC v.1944
Architecture:   64bit
numpy:          C:/Users/stvya/anaconda3/envs/r-tensorflow/Lib/site-packages/numpy
numpy_version:  2.1.3

NOTE: Python version was forced by use_python() function
```

```r
library(keras3)
library(tensorflow)
```

```
Attaching package: 'tensorflow'

The following objects are masked from 'package:keras3':

    set_random_seed, shape
```

```r
library(janitor)
```

```
Attaching package: 'janitor'

The following objects are masked from 'package:stats':

    chisq.test, fisher.test
```

```r
library(lubridate)
```

```
Attaching package: 'lubridate'

The following objects are masked from 'package:base':

    date, intersect, setdiff, union
```

Calling the Jena Climate dataset

```r
library(readr)
path <- "C:/Users/stvya/Downloads/jena_climate_2009_2016.csv"

df_jena <- read_csv(path)
```

```
Rows: 420551 Columns: 15
-- Column specification --------------------------------------------------------
Delimiter: ","
chr  (1): Date Time
dbl (14): p (mbar), T (degC), Tpot (K), Tdew (degC), rh (%), VPmax (mbar), V...

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```r
df_jena <- clean_names(df_jena)
```

```r
tail(df_jena)
```

```
# A tibble: 6 x 15
  date_time  p_mbar t_deg_c tpot_k tdew_deg_c rh_percent v_pmax_mbar v_pact_mbar
  <chr>       <dbl>   <dbl>  <dbl>      <dbl>      <dbl>       <dbl>       <dbl>
1 31.12.201~  1000.   -3.93   269.      -8.09       72.6        4.56        3.31
2 31.12.201~  1000.   -4.05   269.      -8.13       73.1        4.52        3.3
3 31.12.201~  1000.   -3.35   270.      -8.06       69.7        4.77        3.32
4 31.12.201~  1000.   -3.16   270.      -8.21       67.9        4.84        3.28
5 31.12.201~  1000.   -4.23   269.      -8.53       71.8        4.46        3.2
6 01.01.201~  1000.   -4.82   268.      -8.42       75.7        4.27        3.23
# i 7 more variables: v_pdef_mbar <dbl>, sh_g_kg <dbl>, h2oc_mmol_mol <dbl>,
#   rho_g_m_3 <dbl>, wv_m_s <dbl>, max_wv_m_s <dbl>, wd_deg <dbl>
```

```r
str(df_jena)
```

```
spc_tbl_ [420,551 x 15] (S3: spec_tbl_df/tbl_df/tbl/data.frame)
 $ date_time    : chr [1:420551] "01.01.2009 00:10:00" "01.01.2009 00:20:00" "01.01.2009 00:3
 $ p_mbar       : num [1:420551] 997 997 997 997 997 ...
 $ t_deg_c      : num [1:420551] -8.02 -8.41 -8.51 -8.31 -8.27 -8.05 -7.62 -
7.62 -7.91 -8.43 ...
 $ tpot_k       : num [1:420551] 265 265 265 265 265 ...
 $ tdew_deg_c   : num [1:420551] -8.9 -9.28 -9.31 -9.07 -9.04 -8.78 -8.3 -
8.36 -8.73 -9.34 ...
```

```
 $ rh_percent   : num [1:420551] 93.3 93.4 93.9 94.2 94.1 94.4 94.8 94.4 93.8 93.1 ...
 $ v_pmax_mbar  : num [1:420551] 3.33 3.23 3.21 3.26 3.27 3.33 3.44 3.44 3.36 3.23 ...
 $ v_pact_mbar  : num [1:420551] 3.11 3.02 3.01 3.07 3.08 3.14 3.26 3.25 3.15 3 ...
 $ v_pdef_mbar  : num [1:420551] 0.22 0.21 0.2 0.19 0.19 0.19 0.18 0.19 0.21 0.22 ...
 $ sh_g_kg      : num [1:420551] 1.94 1.89 1.88 1.92 1.92 1.96 2.04 2.03 1.97 1.88 ...
 $ h2oc_mmol_mol: num [1:420551] 3.12 3.03 3.02 3.08 3.09 3.15 3.27 3.26 3.16 3.02 ...
 $ rho_g_m_3    : num [1:420551] 1308 1310 1310 1309 1309 ...
 $ wv_m_s       : num [1:420551] 1.03 0.72 0.19 0.34 0.32 0.21 0.18 0.19 0.28 0.59 ...
 $ max_wv_m_s   : num [1:420551] 1.75 1.5 0.63 0.5 0.63 0.63 0.63 0.5 0.75 0.88 ...
 $ wd_deg       : num [1:420551] 152 136 172 198 214 ...
 - attr(*, "spec")=
  .. cols(
  ..   `Date Time` = col_character(),
  ..   `p (mbar)` = col_double(),
  ..   `T (degC)` = col_double(),
  ..   `Tpot (K)` = col_double(),
  ..   `Tdew (degC)` = col_double(),
  ..   `rh (%)` = col_double(),
  ..   `VPmax (mbar)` = col_double(),
  ..   `VPact (mbar)` = col_double(),
  ..   `VPdef (mbar)` = col_double(),
  ..   `sh (g/kg)` = col_double(),
  ..   `H2OC (mmol/mol)` = col_double(),
  ..   `rho (g/m**3)` = col_double(),
  ..   `wv (m/s)` = col_double(),
  ..   `max. wv (m/s)` = col_double(),
  ..   `wd (deg)` = col_double()
  .. )
 - attr(*, "problems")=<externalptr>
```

```r
dim(df_jena)
```

```
[1] 420551     15
```

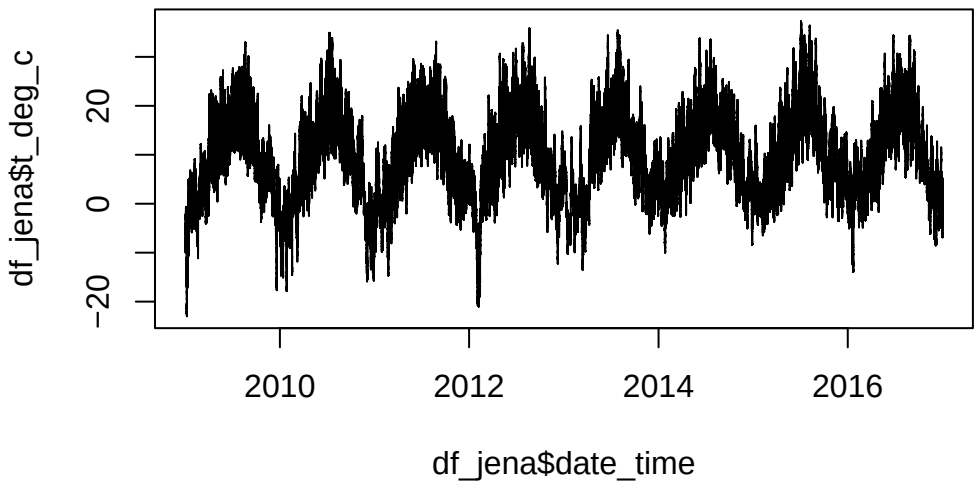
```r
summary(df_jena)
```

```
  date_time             p_mbar          t_deg_c           tpot_k      
 Length:420551      Min.   : 913.6   Min.   :-23.01   Min.   :250.6  
 Class :character   1st Qu.: 984.2   1st Qu.:  3.36   1st Qu.:277.4  
 Mode  :character   Median : 989.6   Median :  9.42   Median :283.5  
                    Mean   : 989.2   Mean   :  9.45   Mean   :283.5  
```

```
                    3rd Qu.: 994.7   3rd Qu.: 15.47   3rd Qu.:289.5
                    Max.   :1015.4   Max.   : 37.28   Max.   :311.3
  tdew_deg_c        rh_percent      v_pmax_mbar     v_pact_mbar
 Min.   :-25.010   Min.   : 12.95   Min.   : 0.95   Min.   : 0.790
 1st Qu.:  0.240   1st Qu.: 65.21   1st Qu.: 7.78   1st Qu.: 6.210
 Median :  5.220   Median : 79.30   Median :11.82   Median : 8.860
 Mean   :  4.956   Mean   : 76.01   Mean   :13.58   Mean   : 9.534
 3rd Qu.: 10.070   3rd Qu.: 89.40   3rd Qu.:17.60   3rd Qu.:12.350
 Max.   : 23.110   Max.   :100.00   Max.   :63.77   Max.   :28.320
  v_pdef_mbar        sh_g_kg       h2oc_mmol_mol     rho_g_m_3
 Min.   : 0.000   Min.   : 0.500   Min.   : 0.80   Min.   :1059
 1st Qu.: 0.870   1st Qu.: 3.920   1st Qu.: 6.29   1st Qu.:1187
 Median : 2.190   Median : 5.590   Median : 8.96   Median :1214
 Mean   : 4.042   Mean   : 6.022   Mean   : 9.64   Mean   :1216
 3rd Qu.: 5.300   3rd Qu.: 7.800   3rd Qu.:12.49   3rd Qu.:1243
 Max.   :46.010   Max.   :18.130   Max.   :28.82   Max.   :1394
     wv_m_s            max_wv_m_s            wd_deg
 Min.   :-9999.000   Min.   :-9999.000   Min.   :  0.0
 1st Qu.:    0.990   1st Qu.:    1.760   1st Qu.:124.9
 Median :    1.760   Median :    2.960   Median :198.1
 Mean   :    1.702   Mean   :    3.057   Mean   :174.7
 3rd Qu.:    2.860   3rd Qu.:    4.740   3rd Qu.:234.1
 Max.   :   28.490   Max.   :   23.500   Max.   :360.0
```

```r
df_jena$date_time <- as.POSIXct(df_jena$date_time,
                                format = "%d.%m.%Y %H:%M:%S")
```

```r
plot(df_jena$date_time, df_jena$t_deg_c, type = "l")
```




```
#Let us visualise a section of the datetime column to understand the time step in the data
#head(df_jena$date_time, 10)
head(df_jena[, c("date_time")])
```

```
# A tibble: 6 x 1
  date_time
  <dttm>
1 2009-01-01 00:10:00
2 2009-01-01 00:20:00
3 2009-01-01 00:30:00
4 2009-01-01 00:40:00
5 2009-01-01 00:50:00
6 2009-01-01 01:00:00
```

We can see that the time stamp is every 10 minutes. From 2009-01-01 00:10:00 GMT and the next step is 2009-01-01 00:20:00 GMT and so on and so forth…

```
#The time space is 10 minutes as can be seen for this 10 rows of data
head(diff(df_jena$date_time), 10)
```

```
Time differences in mins
 [1] 10 10 10 10 10 10 10 10 10 10
```

```r
#Checking for missing values: As we can see, there are no missing values in the dataset. Whic
colSums(is.na(df_jena))
```

```
    date_time        p_mbar       t_deg_c        tpot_k    tdew_deg_c
            0             0             0             0             0
   rh_percent   v_pmax_mbar   v_pact_mbar   v_pdef_mbar       sh_g_kg
            0             0             0             0             0
h2oc_mmol_mol     rho_g_m_3        wv_m_s    max_wv_m_s        wd_deg
            0             0             0             0             0
```

Naive Model (Jena Climate Data):

We begin with a naive model

```r
obs_per_day <- 24 * (60 / 10) #hours * minutes / interval = 144
lookback <- 10 * obs_per_day # days * hours * minutes / interval =1440
step <- 6             # sample every hour (since evry 10 mins, 1hr = 60/10)
delay <- 144          # predict 24 hours ahead (24*6)
batch_size <- 128
```

```r
generator <- function(data, lookback, delay, min_index, max_index,
                      shuffle = FALSE, batch_size = 128, step = 6) {

  data <- as.matrix(data)

  if (is.null(max_index))
    max_index <- nrow(data) - delay - 1

  i <- min_index + lookback

  function() {

    if (shuffle) {
      rows <- sample((min_index + lookback):max_index, size = batch_size)
    } else {
      if (i + batch_size >= max_index)
        i <<- min_index + lookback

      rows <- i:min(i + batch_size - 1, max_index)
      i <<- i + length(rows)
```

```
    }

    timesteps <- length(seq(1, lookback, by = step))

    samples <- array(0, dim = c(length(rows), timesteps, ncol(data)))
    targets <- array(0, dim = c(length(rows)))

    for (j in seq_along(rows)) {

      start <- rows[j] - lookback
      end <- rows[j] - 1

      if (start < 1) next

      indices <- seq(start, end, by = step)

      samples[j,,] <- data[indices, , drop = FALSE]
      targets[j] <- data[rows[j] + delay, 2]
    }

    list(samples, targets)
  }
}
```

```
#convert the data into numeric datatyes; remove datatime, then convert all to numeric, then s

df <- df_jena

# 1. remove datetime
df$date_time <- NULL   # adjust name

# 2. force numeric
df[] <- lapply(df, as.numeric)

# 3. scale (recommended)
df <- scale(df)
```

```
# applying the generator}
train_gen <- generator(df, lookback, delay,
                       min_index = 1,
                       max_index = 200000,
                       shuffle = FALSE,
```

```r
                        batch_size = batch_size,
                        step = step)

batch <- train_gen()
str(batch)
```

```
List of 2
 $ : num [1:128, 1:240, 1:14] 0.874 0.88 0.875 0.873 0.873 ...
 $ : num [1:128(1d)] -2.17 -2.18 -2.21 -2.21 -2.17 ...
```

```r
# Some checks: Printing the shape of a sample batch}
batch <- train_gen()

dim(batch[[1]])
```

```
[1] 128 240  14
```

```r
dim(batch[[2]])
```

```
[1] 128
```

```r
evaluate_naive <- function(generator, steps) {
  batch_maes <- c()

  for (step in 1:steps) {
    batch <- generator()
    samples <- batch[[1]]
    targets <- batch[[2]]

    # last timestep temperature (column 2 assumed temp)
    preds <- samples[, dim(samples)[2], 2]

    mae <- mean(abs(preds - targets))
    batch_maes <- c(batch_maes, mae)
  }

  mean(batch_maes)
}

baseline_mae <- evaluate_naive(train_gen, steps = 100)
baseline_mae
```

```
[1] 0.2979193
```

From the result, our naive model mean absolute error is 0.2979 ~ 0.30

Naive Model (MNIST Data)

```
mnist <- dataset_mnist()
train_images <- mnist$train$x
train_labels <- mnist$train$y
test_images <- mnist$test$x
test_labels <- mnist$test$y
```

```
#Let us flatten it
train_labels <- as.integer(train_labels)
test_labels <- as.integer(test_labels)
```

```
set.seed(260408)

random_preds <- sample(0:9, length(test_labels), replace = TRUE)
```

```
accuracy <- mean(random_preds == test_labels)
round(accuracy * 100,0)
```

```
[1] 10
```

PART 4

Part 0: Data Preparation and Exploratory Analysis

```
#recall the dataframe: df_Jena
#We cast the datetime column into a date time column

df_jena$date_time <- as.POSIXct(df_jena$date_time, format = "%Y-%m-%d %H:%M:%S")
head(df_jena)
```

```
# A tibble: 6 x 15
  date_time           p_mbar t_deg_c tpot_k tdew_deg_c rh_percent v_pmax_mbar
  <dttm>               <dbl>   <dbl>  <dbl>      <dbl>      <dbl>       <dbl>
1 2009-01-01 00:10:00   997.   -8.02   265.      -8.9        93.3        3.33
2 2009-01-01 00:20:00   997.   -8.41   265.      -9.28       93.4        3.23
3 2009-01-01 00:30:00   997.   -8.51   265.      -9.31       93.9        3.21
4 2009-01-01 00:40:00   997.   -8.31   265.      -9.07       94.2        3.26
5 2009-01-01 00:50:00   997.   -8.27   265.      -9.04       94.1        3.27
6 2009-01-01 01:00:00   996.   -8.05   265.      -8.78       94.4        3.33
# i 8 more variables: v_pact_mbar <dbl>, v_pdef_mbar <dbl>, sh_g_kg <dbl>,
#   h2oc_mmol_mol <dbl>, rho_g_m_3 <dbl>, wv_m_s <dbl>, max_wv_m_s <dbl>,
#   wd_deg <dbl>
```

```r
dim(df_jena)
```

```
[1] 420551     15
```

Hourly Resampling of data

```r
library(dplyr)
```

```
Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

    filter, lag

The following objects are masked from 'package:base':

    intersect, setdiff, setequal, union
```

```r
library(lubridate)

df_hourly <- df_jena %>%
  group_by(date_time = floor_date(date_time, "hour")) %>%
  summarise(across(where(is.numeric), mean))
```

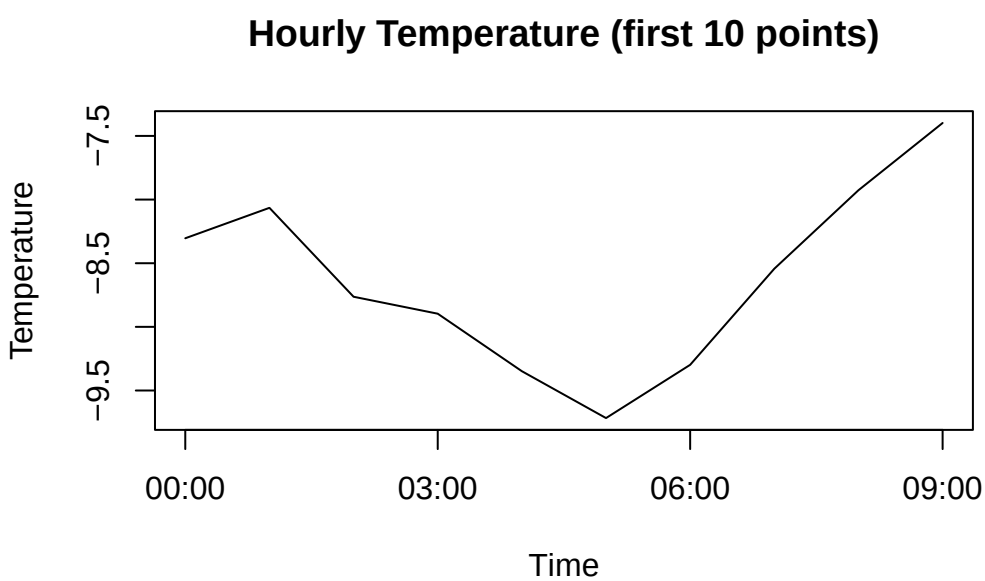
```r
# Shape of hourly resampled data
dim(df_hourly)
```

```
[1] 70041    15
```

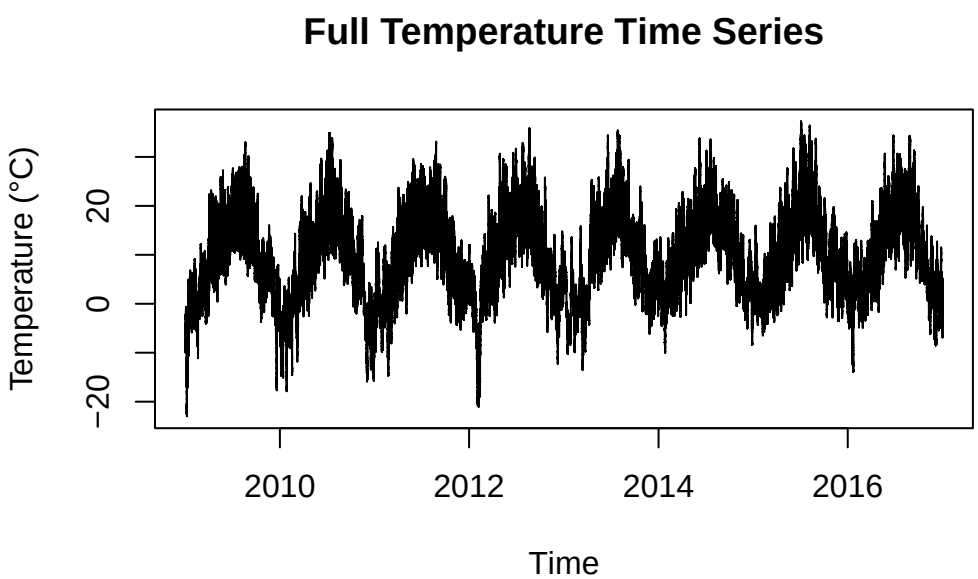
```r
#Checking for Missing values in the dataset
colSums(is.na(df_hourly))
```

```
    date_time        p_mbar       t_deg_c        tpot_k    tdew_deg_c
            0             0             0             0             0
   rh_percent   v_pmax_mbar   v_pact_mbar   v_pdef_mbar       sh_g_kg
            0             0             0             0             0
h2oc_mmol_mol     rho_g_m_3        wv_m_s    max_wv_m_s        wd_deg
            0             0             0             0             0
```

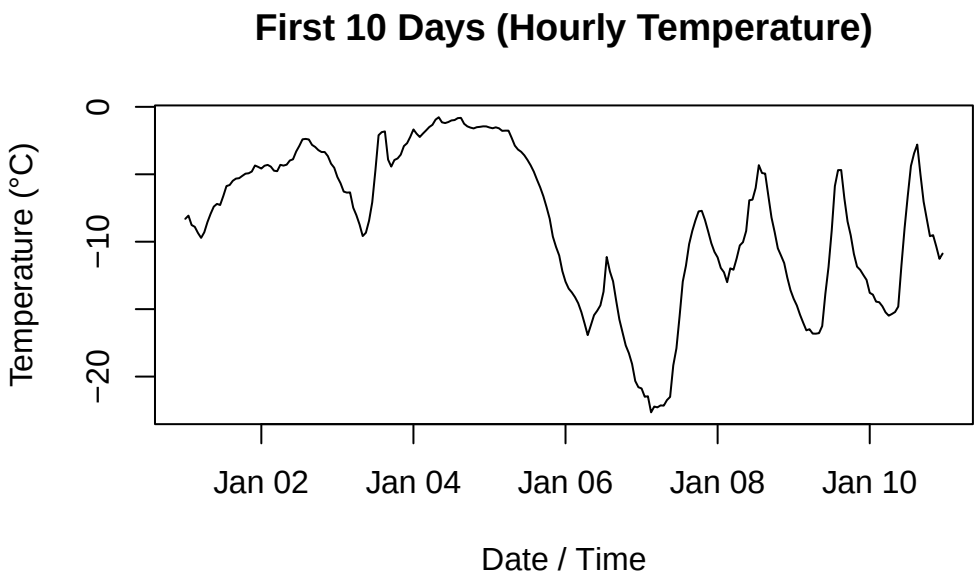
```r
plot(df_hourly$date_time[1:10],
     df_hourly$t_deg_c[1:10],
     type = "l",
     xlab = "Time",
     ylab = "Temperature",
     main = "Hourly Temperature (first 10 points)")
```



```r
#We view the full time series
plot(df_jena$date_time,
     df_jena$t_deg_c,
     type = "l",
     xlab = "Time",
     ylab = "Temperature (°C)",
     main = "Full Temperature Time Series")
```

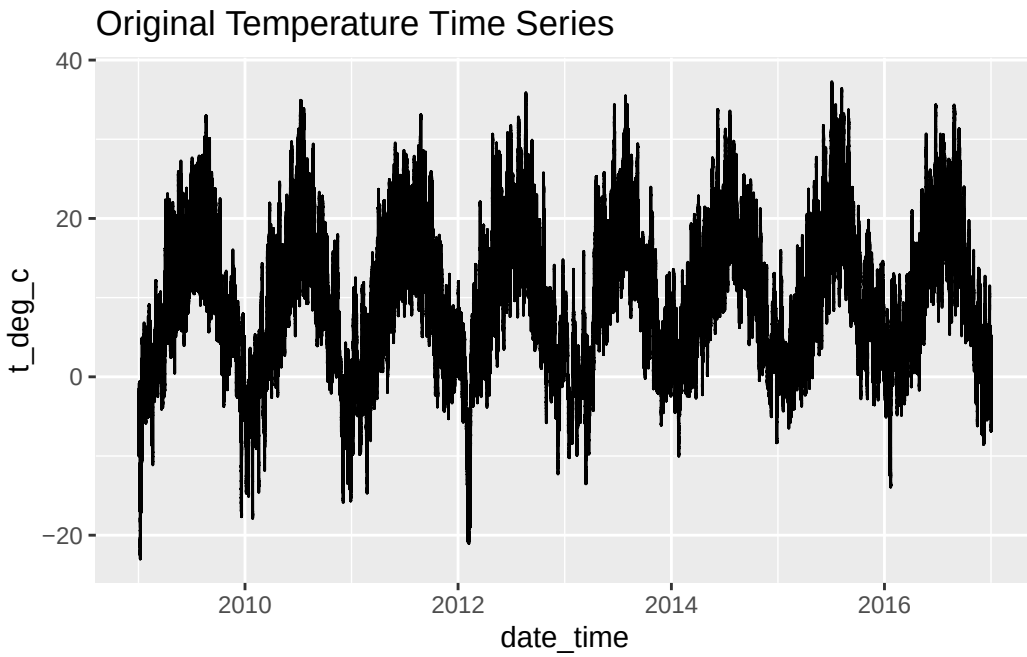



```r
#Plotting the first 10 days for the hourly resampled data
# select first 10 days (10 * 24 hours = 240 points)
df_10_days <- df_hourly[1:(10*24), ]

plot(df_10_days$date_time,
     df_10_days$t_deg_c,
     type = "l",
     xlab = "Date / Time",
     ylab = "Temperature (°C)",
     main = "First 10 Days (Hourly Temperature)")
```

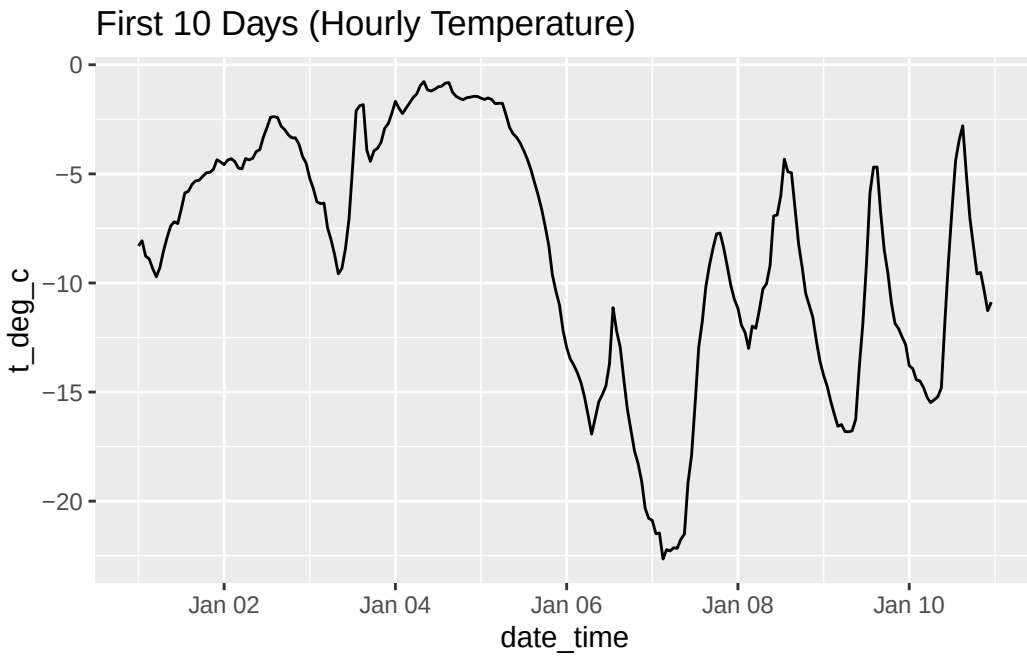


Using ggplot2 to depict the same scenarios

```
library(ggplot2)

# full series 10 minutes interval data
ggplot(df_jena, aes(x = date_time, y = t_deg_c)) +
  geom_line() +
  labs(title = "Original Temperature Time Series")
```




```
# 10-day zoom
ggplot(df_10_days, aes(x = date_time, y = t_deg_c)) +
  geom_line() +
  labs(title = "First 10 Days (Hourly Temperature)")
```



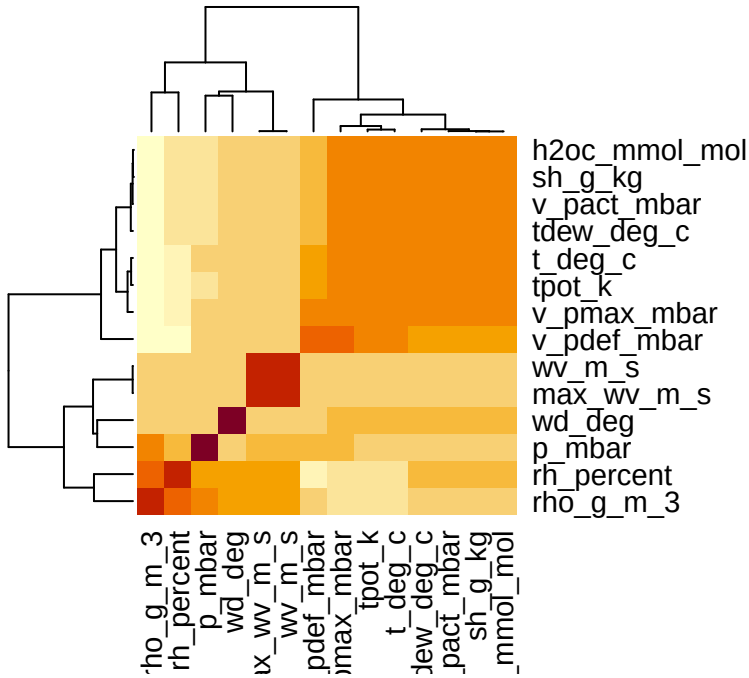

```r
df_num <- df_hourly[, sapply(df_hourly, is.numeric)]

cor_matrix <- cor(df_num, use = "complete.obs")
```

```r
cor_matrix
```

```
                    p_mbar      t_deg_c       tpot_k   tdew_deg_c   rh_percent
p_mbar         1.000000000 -0.045223973 -0.124648322 -0.066924479 -0.019094320
t_deg_c       -0.045223973  1.000000000  0.996820275  0.896855783 -0.572049277
tpot_k        -0.124648322  0.996820275  1.000000000  0.896073106 -0.566701283
tdew_deg_c    -0.066924479  0.896855783  0.896073106  1.000000000 -0.158619744
rh_percent    -0.019094320 -0.572049277 -0.566701283 -0.158619744  1.000000000
v_pmax_mbar   -0.031387571  0.951203880  0.947375513  0.800722304 -0.615609919
v_pact_mbar   -0.054591152  0.869020552  0.867568960  0.968394046 -0.153802457
v_pdef_mbar   -0.002923002  0.762056187  0.757234606  0.438354865 -0.843656613
sh_g_kg       -0.069987978  0.868096721  0.867893138  0.967649033 -0.153136038
h2oc_mmol_mol -0.070029339  0.868516380  0.868312458  0.968093384 -0.153262505
rho_g_m_3      0.307745194 -0.963347488 -0.981318219 -0.886209501  0.513811742
wv_m_s        -0.005968077 -0.004966469 -0.004448357 -0.009140787 -0.005193732
max_wv_m_s    -0.008001097 -0.003061672 -0.002393051 -0.009389937 -0.010094645
wd_deg        -0.074911457  0.046040040  0.051809458  0.060172743 -0.017467086
               v_pmax_mbar v_pact_mbar  v_pdef_mbar      sh_g_kg h2oc_mmol_mol
```

```
p_mbar        -0.031387571 -0.05459115 -0.002923002 -0.069987978  -0.070029339
t_deg_c        0.951203880  0.86902055  0.762056187  0.868096721   0.868516380
tpot_k         0.947375513  0.86756896  0.757234606  0.867893138   0.868312458
tdew_deg_c     0.800722304  0.96839405  0.438354865  0.967649033   0.968093384
rh_percent    -0.615609919 -0.15380246 -0.843656613 -0.153136038  -0.153262505
v_pmax_mbar    1.000000000  0.82658659  0.875781640  0.826176355   0.826206602
v_pact_mbar    0.826586590  1.00000000  0.452237076  0.999851803   0.999856047
v_pdef_mbar    0.875781640  0.45223708  1.000000000  0.451714107   0.451758379
sh_g_kg        0.826176355  0.99985180  0.451714107  1.000000000   0.999997985
h2oc_mmol_mol  0.826206602  0.99985605  0.451758379  0.999997985   1.000000000
rho_g_m_3     -0.901596779 -0.85140554 -0.698551496 -0.854489639  -0.854931928
wv_m_s        -0.004262852 -0.01007143  0.001882014 -0.009945046  -0.009942456
max_wv_m_s    -0.002391084 -0.01065823  0.005351686 -0.010499661  -0.010495032
wd_deg        -0.011279319  0.02315989 -0.037735994  0.024296915   0.024567405
                rho_g_m_3       wv_m_s   max_wv_m_s      wd_deg
p_mbar         0.307745194 -0.005968077 -0.008001097 -0.07491146
t_deg_c       -0.963347488 -0.004966469 -0.003061672  0.04604004
tpot_k        -0.981318219 -0.004448357 -0.002393051  0.05180946
tdew_deg_c    -0.886209501 -0.009140787 -0.009389937  0.06017274
rh_percent     0.513811742 -0.005193732 -0.010094645 -0.01746709
v_pmax_mbar   -0.901596779 -0.004262852 -0.002391084 -0.01127932
v_pact_mbar   -0.851405538 -0.010071432 -0.010658233  0.02315989
v_pdef_mbar   -0.698551496  0.001882014  0.005351686 -0.03773599
sh_g_kg       -0.854489639 -0.009945046 -0.010499661  0.02429691
h2oc_mmol_mol -0.854931928 -0.009942456 -0.010495032  0.02456741
rho_g_m_3      1.000000000  0.003442811  0.001208068 -0.06900978
wv_m_s         0.003442811  1.000000000  0.992596895 -0.01822001
max_wv_m_s     0.001208068  0.992596895  1.000000000 -0.01693987
wd_deg        -0.069009785 -0.018220007 -0.016939871  1.00000000
```

```r
#We can plot the heatmap to visualise the correlation between the features
heatmap(cor_matrix)
```



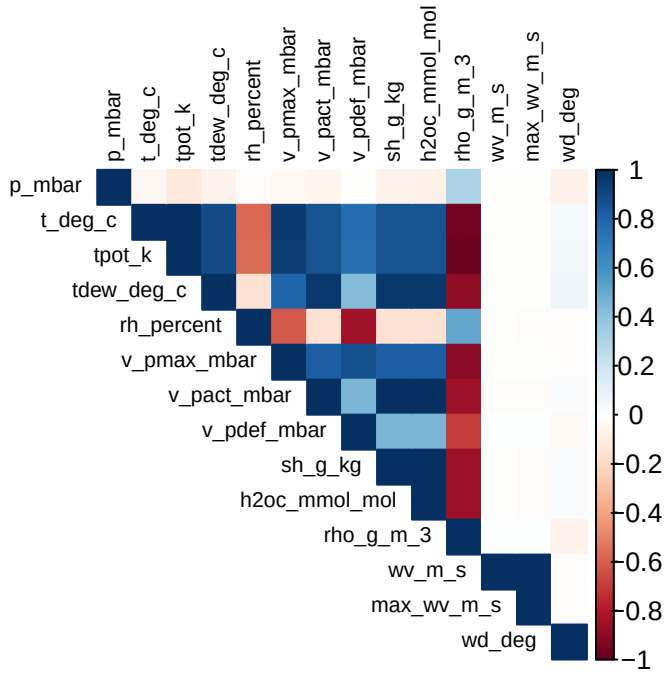

```r
install.packages("corrplot")  # run once
```

```
The following package(s) will be installed:
- corrplot [0.95]
These packages will be installed into "~/DNN project1/renv/library/windows/R-
4.5/x86_64-w64-mingw32".

# Installing packages --------------------------------------------------------
- Installing corrplot 0.95 ...                  OK [linked from cache]
Successfully installed 1 package in 20 milliseconds.
```

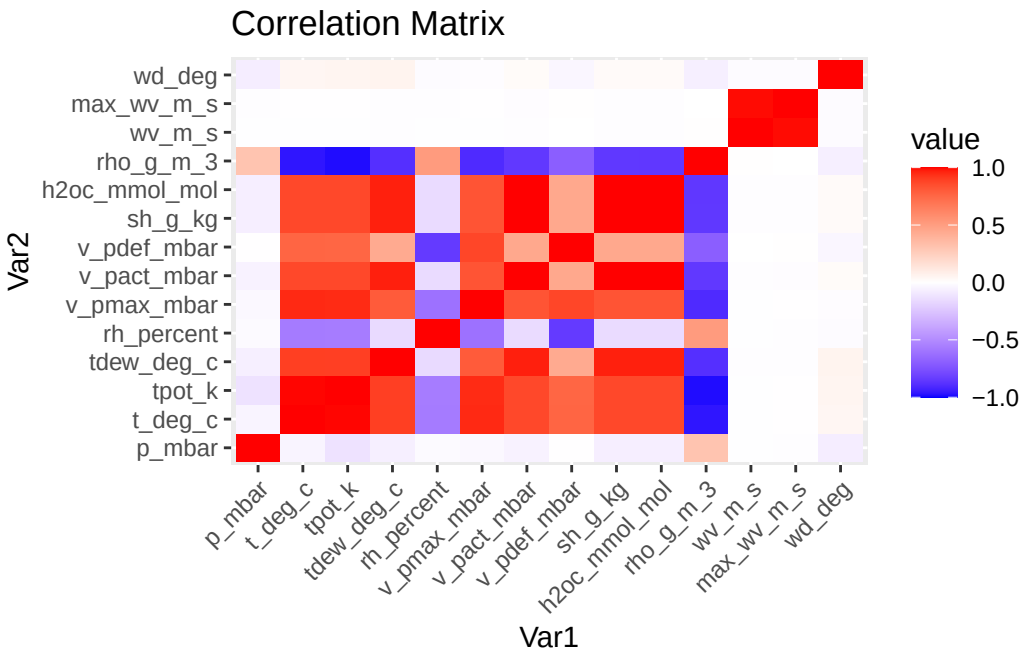
```r
library(corrplot)
```

```
corrplot 0.95 loaded
```

```r
corrplot(cor_matrix,
         method = "color",
         type = "upper",
         tl.col = "black",
         tl.cex = 0.7)
```




```r
library(reshape2)
library(ggplot2)

cor_melt <- melt(cor_matrix)

ggplot(cor_melt, aes(Var1, Var2, fill = value)) +
  geom_tile() +
  scale_fill_gradient2(low = "blue", high = "red", mid = "white",
                       midpoint = 0, limit = c(-1,1)) +
  theme(axis.text.x = element_text(angle = 45, hjust = 1)) +
  labs(title = "Correlation Matrix")
```




```r
df_num <- df_hourly[, sapply(df_hourly, is.numeric)]
cor_matrix <- cor(df_num, use = "complete.obs")

# convert matrix to long format
cor_df <- as.data.frame(as.table(cor_matrix))

# rename columns
colnames(cor_df) <- c("Var1", "Var2", "Correlation")

# remove self-correlations
cor_df <- cor_df[cor_df$Var1 != cor_df$Var2, ]

# remove duplicate pairs (A-B and B-A)
cor_df <- cor_df[!duplicated(t(apply(cor_df[,1:2], 1, sort))), ]

# filter strong correlations
high_corr <- subset(cor_df, abs(Correlation) > 0.8)

# view results
high_corr <- high_corr[order(-abs(high_corr$Correlation)), ]
high_corr
```

```
             Var1          Var2 Correlation
```

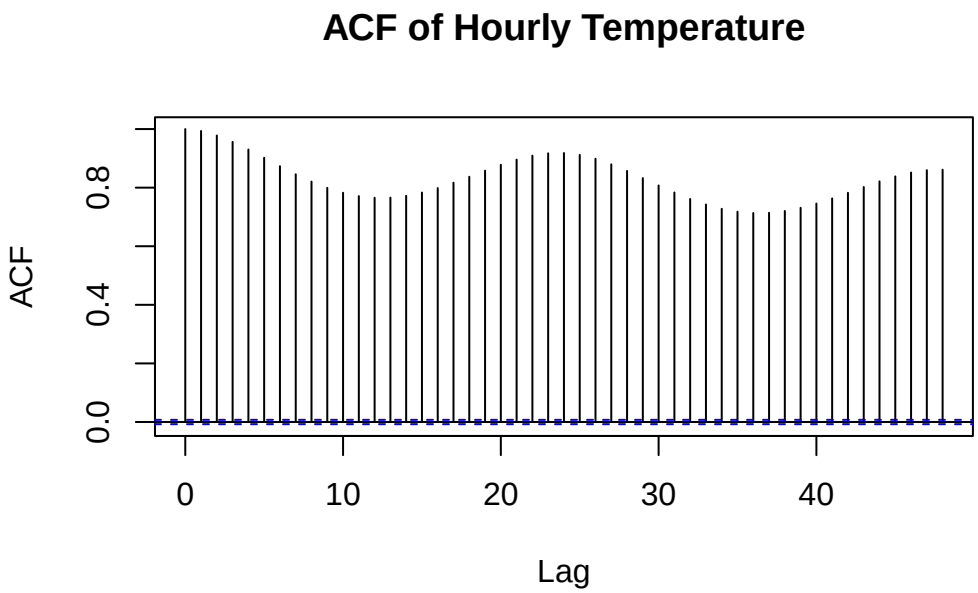
```
122 h2oc_mmol_mol       sh_g_kg   0.9999980
94  h2oc_mmol_mol   v_pact_mbar   0.9998560
93        sh_g_kg   v_pact_mbar   0.9998518
17         tpot_k       t_deg_c   0.9968203
167    max_wv_m_s        wv_m_s   0.9925969
39      rho_g_m_3        tpot_k  -0.9813182
49    v_pact_mbar    tdew_deg_c   0.9683940
52  h2oc_mmol_mol    tdew_deg_c   0.9680934
51        sh_g_kg    tdew_deg_c   0.9676490
25      rho_g_m_3       t_deg_c  -0.9633475
20    v_pmax_mbar       t_deg_c   0.9512039
34    v_pmax_mbar        tpot_k   0.9473755
81      rho_g_m_3   v_pmax_mbar  -0.9015968
18     tdew_deg_c       t_deg_c   0.8968558
32     tdew_deg_c        tpot_k   0.8960731
53      rho_g_m_3    tdew_deg_c  -0.8862095
78    v_pdef_mbar   v_pmax_mbar   0.8757816
21    v_pact_mbar       t_deg_c   0.8690206
24  h2oc_mmol_mol       t_deg_c   0.8685164
38  h2oc_mmol_mol        tpot_k   0.8683125
23        sh_g_kg       t_deg_c   0.8680967
37        sh_g_kg        tpot_k   0.8678931
35    v_pact_mbar        tpot_k   0.8675690
137     rho_g_m_3 h2oc_mmol_mol  -0.8549319
123     rho_g_m_3       sh_g_kg  -0.8544896
95      rho_g_m_3   v_pact_mbar  -0.8514055
64    v_pdef_mbar    rh_percent  -0.8436566
77    v_pact_mbar   v_pmax_mbar   0.8265866
80  h2oc_mmol_mol   v_pmax_mbar   0.8262066
79        sh_g_kg   v_pmax_mbar   0.8261764
48    v_pmax_mbar    tdew_deg_c   0.8007223
```

The correlation is high among these variables because, it appears temperature related variables are highly correlated.

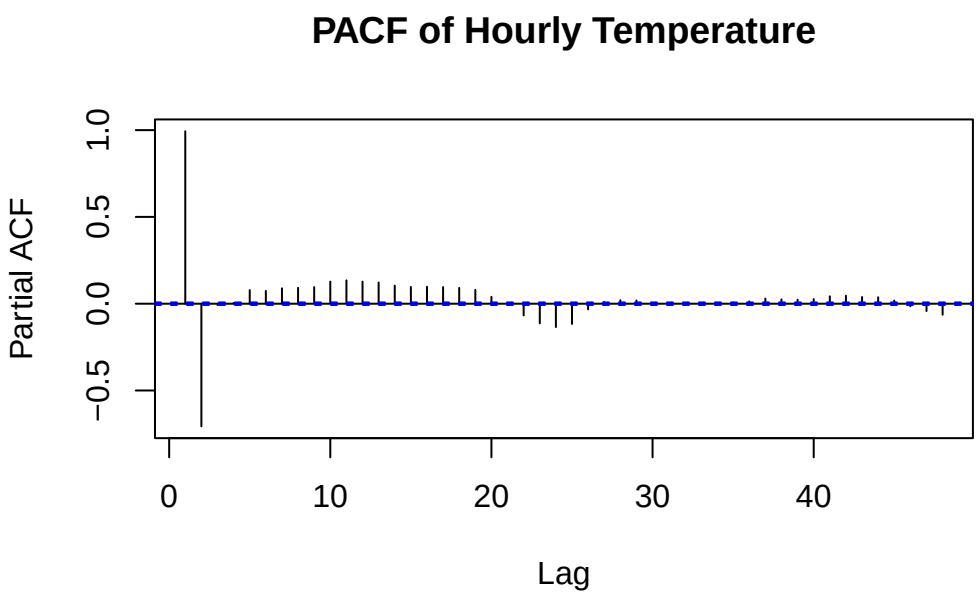
Plotting ACF and PACF for the timeseries data

```r
# We extract the temperature data from the hourly data
temp <- df_hourly$t_deg_c
```

```r
acf(temp,
    main = "ACF of Hourly Temperature")
```




```r
pacf(temp,
     main = "PACF of Hourly Temperature")
```



Interpretation ACF

- Observe high spikes at onset and drop at 12, 24, etc hours
- Slow decay
- Possible spike around 12, 24, 48, …
- It can be translated to mean past several days still influence current temperature

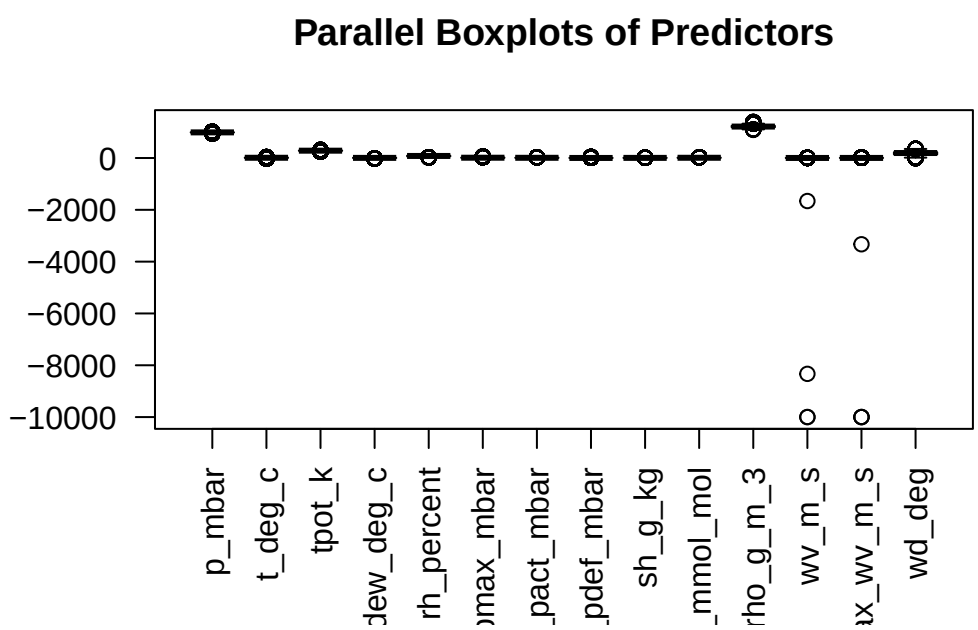
Observations and Interpretation of PACF

- strong spikes at early lags
- then sharp drop maybe a smaller spike around 24 hours
- Can be translated to mean, direct influence is mostly short-term, but daily cycles matter

120-hour lookback

- 120- hour loopback means 5 days of history,; This means full daily cycles (24h), multiple cycles (5 repetitions) and medium-term weather persistence

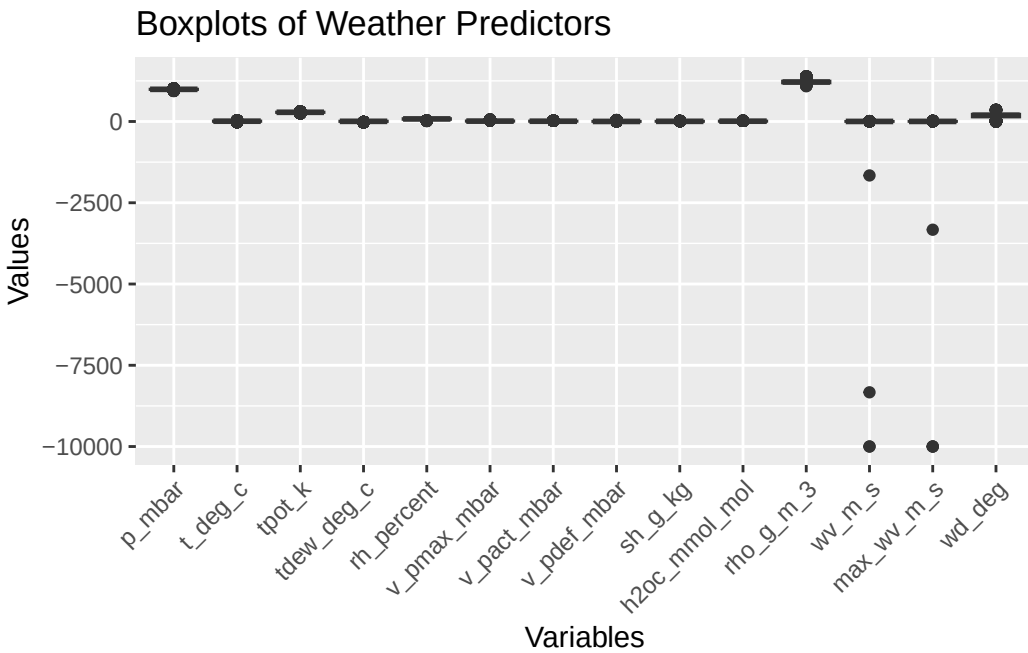
```r
df_num <- df_hourly[, sapply(df_hourly, is.numeric)]

boxplot(df_num,
        main = "Parallel Boxplots of Predictors",
        las = 2,        # rotate labels
        col = "lightblue")
```




```r
#Using ggplot2
library(ggplot2)
library(reshape2)

df_num <- df_hourly[, sapply(df_hourly, is.numeric)]

df_long <- melt(df_num)
```

```
No id variables; using all as measure variables
```

```r
ggplot(df_long, aes(x = variable, y = value)) +
  geom_boxplot(fill = "lightblue") +
  labs(x = "Variables", y = "Values",
       title = "Boxplots of Weather Predictors") +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```



The parallel boxplots show significant differences in the scale and distribution of predictor variables, with some features spanning vastly different numeric ranges. This indicates that normalization is necessary to prevent features with larger magnitudes from dominating the learning process. Standardization (z-score scaling) is therefore appropriate and should be applied prior to model training.

Data Spliting

```
#sorting the data
library(dplyr)

# 1. Sort time series
df <- df_hourly[order(df_hourly$date_time), ]
```

Building a Generator

```
generator <- function(data,
                      lookback = 120,
                      delay = 24,
```

```
                     step = 1,
                     batch_size = 128,
                     shuffle = FALSE,
                     task = c("regression", "classification")) {

task <- match.arg(task)

min_index <- lookback + 1
max_index <- nrow(data) - delay

i <- min_index

function() {

  if (shuffle) {

    rows <- sample(min_index:max_index, size = batch_size)

  } else {

    if (i + batch_size > max_index) {
      i <<- min_index
    }

    rows <- i:min(i + batch_size - 1, max_index)
    i <<- i + length(rows)
  }


  rows <- rows[rows + delay <= nrow(data)]

  n_features <- ncol(data)

  samples <- array(0,
                   dim = c(length(rows),
                           lookback / step,
                           n_features))

  targets <- vector("list", length(rows))

  for (j in seq_along(rows)) {
```

```
      idx <- seq(rows[j] - lookback,
                 rows[j] - 1,
                 by = step)
      samples[j, , ] <- as.matrix(data[idx, feature_cols])

      samples[j, , ] <- data[idx, ]

      if (task == "regression") {

        targets[[j]] <- data[rows[j] + delay, 1]

      } else {

        targets[[j]] <- data[rows[j] + delay, "class"]
      }
    }

    list(samples, unlist(targets))
  }
}
```

```
generator <- function(data,
                      lookback = 120,
                      delay = 24,
                      step = 1,
                      batch_size = 128,
                      shuffle = FALSE,
                      task = c("regression", "classification")) {

  task <- match.arg(task)

  min_index <- lookback + 1
  max_index <- nrow(data) - delay

  i <- min_index

  function() {

    if (shuffle) {
      rows <- sample(min_index:max_index, size = batch_size)
    } else {
      if (i + batch_size >= max_index)
```

```
        i <<- min_index

      rows <- i:min(i + batch_size - 1, max_index)
      i <<- i + length(rows)
    }

    # IMPORTANT: exclude class column from inputs
    feature_cols <- setdiff(names(data), "class")

    n_features <- length(feature_cols)

    samples <- array(0,
                     dim = c(length(rows),
                             lookback,
                             n_features))

    targets <- numeric(length(rows))

    for (j in seq_along(rows)) {

      idx <- seq(rows[j] - lookback,
                 rows[j] - 1,
                 by = step)

      #  FIX HERE
      samples[j, , ] <- as.matrix(data[idx, feature_cols])

      if (task == "classification") {
        targets[j] <- as.integer(data[rows[j] + delay, "class"])
      } else {
        targets[j] <- data[rows[j] + delay, "target_temp"]
      }
    }

    list(samples, targets)
  }
}
```

Regression Pipeline::

```
df$target_temp <- dplyr::lead(df$t_deg_c, 24)
df_reg <- na.omit(df)
```

```
#Time split with no shuffle
#Split the data chronologically into:
# Training: Jan 2009 - Dec 2014
# Validation: Jan 2015 - Dec 2015
# Test: Jan 2016 - Dec 2016
train_df <- subset(df_reg,
                   date_time >= as.POSIXct("2009-01-01") &
                   date_time <= as.POSIXct("2014-12-31 23:59:59"))

val_df <- subset(df_reg,
                 date_time >= as.POSIXct("2015-01-01") &
                 date_time <= as.POSIXct("2015-12-31 23:59:59"))

test_df <- subset(df_reg,
                  date_time >= as.POSIXct("2016-01-01") &
                  date_time <= as.POSIXct("2016-12-31 23:59:59"))
```

```
#We normalize the train set
train_x <- train_df[, sapply(train_df, is.numeric)]
val_x   <- val_df[, sapply(val_df, is.numeric)]
test_x  <- test_df[, sapply(test_df, is.numeric)]

train_mean <- sapply(train_x, mean)
train_sd   <- sapply(train_x, sd)

train_scaled <- scale(train_x, center = train_mean, scale = train_sd)
val_scaled   <- scale(val_x, center = train_mean, scale = train_sd)
test_scaled  <- scale(test_x, center = train_mean, scale = train_sd)
```

```
train_y <- train_df$target_temp
val_y   <- val_df$target_temp
test_y  <- test_df$target_temp
```

```
train_gen_reg <- generator(train_scaled,
                           lookback = 120,
```

```
                              delay = 24,
                              step = 1,
                              batch_size = 128,
                              shuffle = TRUE,
                              task = "regression")
```

```
batch <- train_gen_reg()

dim(batch[[1]])
```

```
[1] 128 120   0
```

```
length(batch[[2]])
```

```
[1] 128
```

Classification Pipeline

```
df$class <- cut(df$t_deg_c,
                breaks = quantile(df$t_deg_c,
                                  probs = seq(0, 1, 0.25),
                                  na.rm = TRUE),
                labels = c("Cold", "Cool", "Mild", "Warm"),
                include.lowest = TRUE)

df_cls <- df
```

```
train_df <- subset(df_cls,
                   date_time >= as.POSIXct("2009-01-01") &
                   date_time <= as.POSIXct("2014-12-31 23:59:59"))

val_df <- subset(df_cls,
                 date_time >= as.POSIXct("2015-01-01") &
                 date_time <= as.POSIXct("2015-12-31 23:59:59"))

test_df <- subset(df_cls,
                  date_time >= as.POSIXct("2016-01-01") &
                  date_time <= as.POSIXct("2016-12-31 23:59:59"))
```

```r
train_x <- train_df[, sapply(train_df, is.numeric)]
val_x   <- val_df[, sapply(val_df, is.numeric)]
test_x  <- test_df[, sapply(test_df, is.numeric)]

train_mean <- sapply(train_x, mean)
train_sd   <- sapply(train_x, sd)

train_scaled <- scale(train_x, center = train_mean, scale = train_sd)
val_scaled   <- scale(val_x, center = train_mean, scale = train_sd)
test_scaled  <- scale(test_x, center = train_mean, scale = train_sd)
```

```r
train_scaled <- as.data.frame(train_scaled)
val_scaled   <- as.data.frame(val_scaled)
test_scaled  <- as.data.frame(test_scaled)


train_scaled$class <- train_df$class
val_scaled$class   <- val_df$class
test_scaled$class  <- test_df$class
```

```r
train_y <- train_df$class
val_y   <- val_df$class
test_y  <- test_df$class
```

```r
train_gen_cls <- generator(train_scaled,
                           lookback = 120,
                           delay = 24,
                           step = 1,
                           batch_size = 128,
                           shuffle = TRUE,
                           task = "classification")
```

```r
batch <- train_gen_cls()

dim(batch[[1]])
```

```
[1] 128 120  15
```

```r
length(batch[[2]])
```

```
[1] 128
```